

Virtual Smart Water & Flood Early-Warning Gateway

Dương Hữu Huỳnh — 20251197M

Nông Đức Huy — MSSV

Bùi Phạm Sơn Hà — MSSV

June 24, 2026

Mục lục

1. Giới thiệu	3
1.1. Bối cảnh và động lực	3
1.2. Mục tiêu	3
1.3. Phạm vi và cách tiếp cận	3
2. Kiến trúc hệ thống	4
2.1. Các nhóm thành phần	4
3. Thiết kế topic và bản tin	5
3.1. Phân cấp topic MQTT	5
3.2. Định dạng bản tin	5
4. Luồng Uplink — từ cảm biến lên cloud	6
4.1. Mô phỏng cảm biến: một cơn bão chung trên trục thời gian	6
4.2. Chuẩn hóa tại gateway	7
5. Rule engine	8
5.1. Bốn luật bắt buộc	8
6. Luồng Downlink — điều khiển thiết bị	9
6.1. Thiết bị actuator	9
7. Tích hợp ThingsBoard (uplink + RPC)	10
7.1. Điều khiển từ xa: RPC → lệnh cục bộ	10
7.2. Chính ngưỡng từ xa bằng shared attributes	10
7.3. Dashboard, bản đồ và alarm trên ThingsBoard	11
8. Lưu trữ và giám sát	12
8.1. Lược đồ InfluxDB	12
8.2. Dashboard Grafana	12
8.3. REST API (FastAPI)	13
9. Triển khai Docker Compose	14
9.1. Tham số hóa qua biến môi trường	14
9.2. Vận hành	15
9.3. Khả năng phục hồi	15
10. Trả lời các câu hỏi bắt buộc	15
11. Đánh giá và kết luận	16
11.1. Phân công công việc	16
11.2. Các yêu cầu nâng cao	17
11.3. Hạn chế và hướng phát triển	17
11.4. Kết luận	17
Danh mục hình	18
Danh mục bảng	18

1. Giới thiệu

1.1. Bối cảnh và động lực

Ngập lụt đô thị và lũ trên sông gây thiệt hại lớn về người và tài sản. Trong các tình huống này, thời gian là yếu tố sống còn: cảnh báo sớm dù chỉ vài phút cũng đủ để vận hành bơm tiêu úng, mở cửa xả, hù còi sơ tán và giảm đáng kể thiệt hại. Một hệ thống quan trắc hiện đại thường đặt cảm biến tại nhiều trạm dọc sông hoặc kênh tiêu để theo dõi mực nước, lưu lượng và lượng mưa theo thời gian thực.

Điểm mấu chốt là khả năng phản ứng tại chỗ: một gateway đặt tại biên (edge) có thể phát hiện mực nước vượt ngưỡng và lập tức kích hoạt actuator mà không phụ thuộc vào đường truyền lên server. Song song đó, dữ liệu vẫn được đẩy lên server để trung tâm điều khiển theo dõi toàn lưu vực và ra lệnh từ xa khi cần.

1.2. Mục tiêu

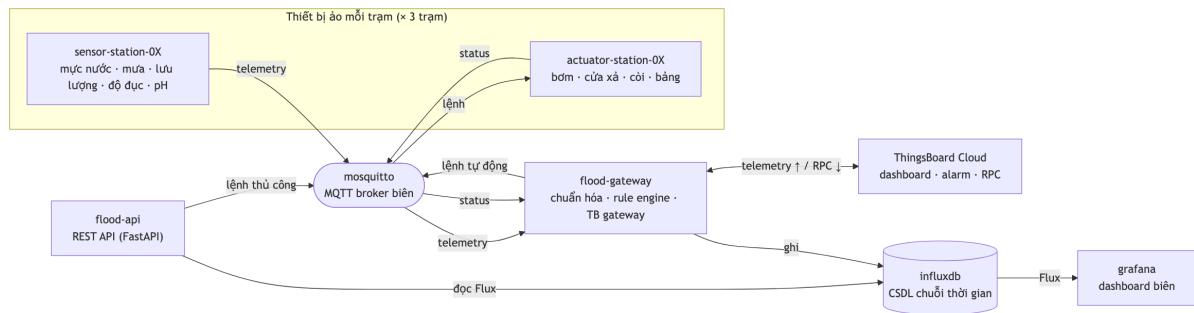
Mini-project triển khai toàn bộ hệ thống ở mức phần mềm — mọi thiết bị vật lý được giả lập bằng Python trong container Docker — với các mục tiêu:

- Lập trình nhiều virtual sensor mô phỏng các trạm quan trắc khác nhau và một edge gateway nhận telemetry MQTT từ nhiều trạm.
- Thiết kế topic hierarchy và message format cho quan trắc nước; chuẩn hóa dữ liệu (mực nước, lưu lượng, lượng mưa, độ đục, pH).
- Phát hiện cảnh báo nhiều mức (advisory / warning / emergency) bằng rule engine và tự động điều khiển bơm, cửa xả, còi, bảng cảnh báo qua MQTT.
- Lưu telemetry – event – trạng thái vào InfluxDB; giám sát bằng Grafana và ThingsBoard; cung cấp REST API truy vấn và điều khiển thủ công.
- Tích hợp ThingsBoard: đẩy telemetry nhiều trạm (uplink) và nhận RPC điều khiển từ xa (downlink). Đóng gói và triển khai bằng Docker Compose.

1.3. Phạm vi và cách tiếp cận

Hệ thống mô phỏng một lưu vực gồm ba trạm `station-01`, `station-02`, `station-03` (đặt theo các trạm thực trên sông Hồng: Yên Bái → Sơn Tây → Hà Nội). Mỗi trạm có một sensor node và một actuator node. Đặc trưng cốt lõi của đề tài là kiến trúc lai: xử lý thời gian thực tại biên cho các tình huống khẩn cấp, kết hợp giám sát toàn lưu vực trên cloud.

2. Kiến trúc hệ thống



Hình 1: Kiến trúc tổng thể: thiết bị ảo mỗi trạm, broker MQTT, edge gateway, InfluxDB, Grafana, REST API và ThingsBoard.

2.1. Các nhóm thành phần

Hệ thống đầy đủ các nhóm chức năng mà một hệ IoT cần: *mô phỏng cảm biến, mô phỏng thiết bị actuator, broker, gateway, cơ sở dữ liệu, dashboard* và *API*. Bảng Bảng 1 liệt kê 11 container và vai trò của chúng.

Dịch vụ	Vai trò
mosquitto	MQTT broker biên — trực giao tiếp nội bộ.
sensor-station-01/02/03	Cảm biến ảo mỗi trạm: mức nước, mưa, lưu lượng, độ đục, pH.
actuator-station-01/02/03	Thiết bị actuator ảo: bơm, cửa xả, còi, bảng cảnh báo.
flood-gateway	Gateway: chuẩn hóa, rule engine, ghi InfluxDB, ThingsBoard Gateway.
influxdb	CSDL chuỗi thời gian (telemetry, event, trạng thái).
grafana	Dashboard giám sát tại biên (auto-provisioned).
flood-api	REST API (FastAPI): truy vấn trạng thái + gửi lệnh thủ công.
ThingsBoard	Đám mây: dashboard, bản đồ, alarm nhiều mức, RPC.

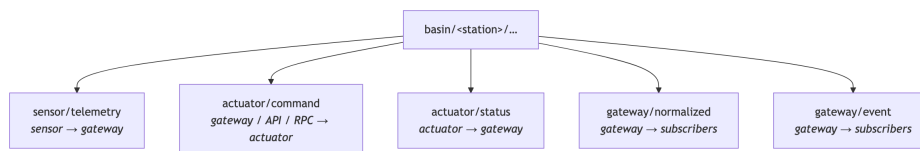
Bảng 1: Bản đồ dịch vụ — 11 container trong một dự án Docker Compose.

Mỗi tầng giải quyết một bài toán khác nhau. Tầng biên tối ưu cho độ trễ thấp và tính tự chủ: gateway nằm cùng mạng với cảm biến/thiết bị nên luồng “đo → quyết định → điều khiển” khép kín trong mili-giây và không phụ thuộc Internet. Tầng cloud tối ưu cho tầm nhìn toàn cục: tổng hợp nhiều trạm, dashboard tập trung, cảnh báo cho người vận hành và điều khiển từ xa. Cảm biến và thiết bị actuator chỉ giao tiếp qua MQTT cục bộ.

3. Thiết kế topic và bản tin

3.1. Phân cấp topic MQTT

Mọi lưu lượng tại biên dùng lược đồ phân cấp `basin/<station>/<role>/<leaf>`. Lược đồ này mở rộng theo từng trạm và giúp đăng ký wildcard dễ dàng — gateway chỉ cần subscribe `basin/+<station>/telemetry` là nhận được telemetry của mọi trạm.



Hình 2: Cây topic dưới mỗi trạm và chiều đi của bản tin.

Topic	Producer → Consumer
basin/<station>/sensor/telemetry	sensor → gateway
basin/<station>/actuator/command	gateway / API / RPC → actuator
basin/<station>/actuator/status	actuator → gateway
basin/<station>/gateway/normalized	gateway → subscribers
basin/<station>/gateway/event	gateway → subscribers

Bảng 2: Năm topic tại biên cùng cặp producer → consumer.

Các topic `v1/gateway/*` là một kết nối **riêng** tới broker của ThingsBoard, không thuộc broker biên (xem Chương 7).

3.2. Định dạng bản tin

Tất cả bản tin là JSON. Bốn loại bản tin chính:

Telemetry (sensor → gateway):

```
{
  "device_id": "sensor-station-01",
  "basin_id": "red-river",
  "station_id": "station-01",
  "station_name": "Yên Bái",
  "latitude": 21.705,
  "longitude": 104.869,
  "water_level": 3.42,
  "flow_rate": 12.7,
  "rainfall": 45.0,
  "turbidity": 120.0,
  "ph": 7.1,
  "timestamp": "2026-06-10T10:00:00Z"
}
```

Command (gateway/API/RPC → actuator):

```
{
  "station_id": "station-01",
  "target": "pump",
  "action": "on",
  "reason": "water_level_high",
  "timestamp": "2026-06-10T10:00:05Z"
}
```

Event (gateway → subscribers):

```
{
  "station_id": "station-01",
  "event_type": "flood_warning",
  "severity": "warning",
  "value": 3.42,
  "threshold": 3.0,
  "action_taken":
    "pump_on,gate_open,board_warning",
  "timestamp": "2026-06-10T10:00:05Z"
}
```

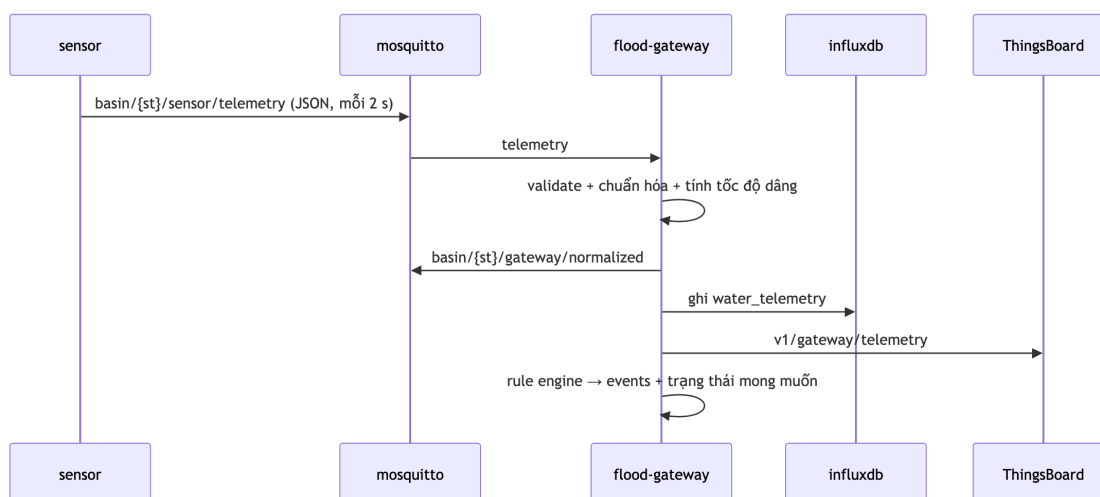
Status (actuator → gateway) mang **toàn bộ** trạng thái thiết bị mỗi lần phát:

```
{
  "device_id": "actuator-station-01", "station_id": "station-01",
  "pump": "on", "gate": "open", "siren": "off", "board": "warning",
  "last_command_reason": "water_level_high", "timestamp": "2026-06-10T10:00:06Z"
}
```

Trong bản tin telemetry, ba trường `station_name`, `latitude`, `longitude` là **mở rộng** ngoài tập tối thiểu của đề bài — chúng cấp dữ liệu cho widget bản đồ trên ThingsBoard; gateway chấp nhận và bỏ qua chúng khi không cần.

4. Luồng Uplink — từ cảm biến lên cloud

Uplink là hành trình của một telemetry: sinh ra tại sensor, đi qua broker, được gateway chuẩn hóa và đánh giá, rồi vừa lưu xuống InfluxDB vừa đẩy lên ThingsBoard (Hình 3).

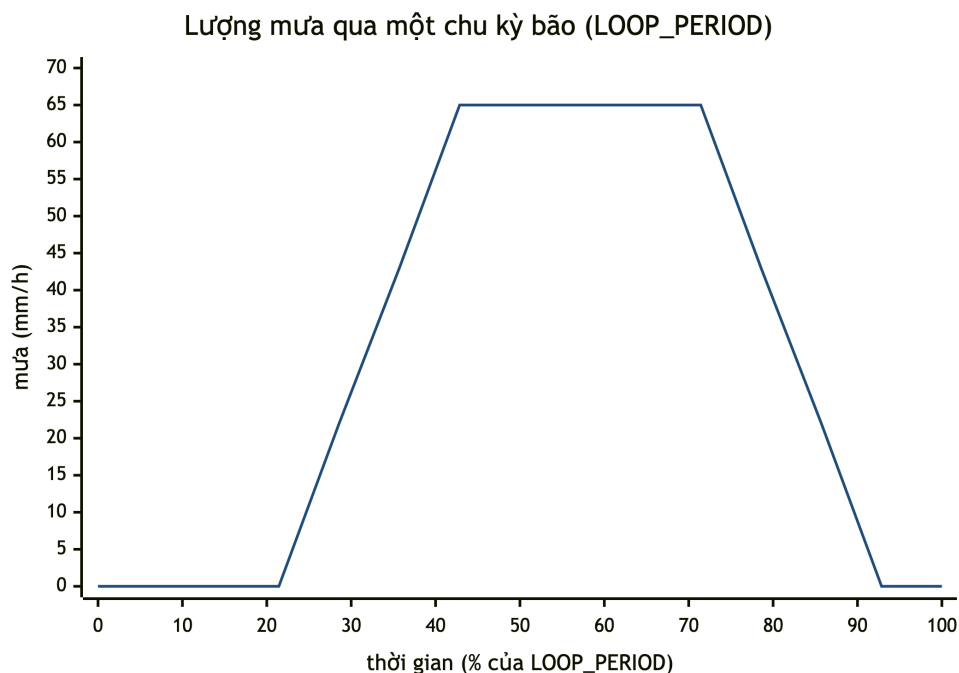


Hình 3: Trình tự uplink: telemetry → chuẩn hóa → lưu trữ + đẩy cloud + rule engine.

4.1. Mô phỏng cảm biến: một cơn bão chung trên trục thời gian

Hệ thống dựng **kịch bản lưu vực tương quan**, trong đó **một** cơn bão duy nhất chi phối cả ba trạm và **sóng lũ lan dần về hạ lưu** đồng thời dâng cao hơn — đúng câu chuyện cảnh báo sớm kinh điển (trạm thượng lưu dâng trước khi thành phố ngập).

Cường độ bão trong một chu kỳ `LOOP_PERIOD` là một hình thang: lặng → dâng → cao nguyên → rút (Hình 4).



Hình 4: Lượng mưa theo hình thang trong một chu kỳ bão (lặng 30% → dâng 15% → cao nguyên 30% → rút 25%).

Mỗi trạm đọc cơn bão đó **trễ** đi một khoảng `lag` (thời gian truyền sóng, tính theo **phần trăm** của `LOOP_PERIOD`), và **tích phân** mưa trừ thoát nước — đây chính là “xu hướng”:

```
phase    = (time.time() - SCENARIO_EPOCH - lag) % LOOP_PERIOD
rainfall = PEAK_RAIN * storm_intensity(phase) + gust + noise
# tích phân: mưa làm dâng, thoát nước kéo mực nước về base
level    += gain * rainfall - DRAINAGE_K * (level - base)
```

Hệ số `gain` được **suy ra** từ đỉnh mong muốn ($gain = (peak - base) \cdot K / PEAK_RAIN$), nên người dùng chỉ cần chỉnh `peak` của mỗi trạm. Kích bản mặc định là **bão lớn**: cả ba trạm đều vượt ngưỡng **khẩn cấp** 4.0 m, lệch pha về hạ lưu (Bảng 3).

Trạm	Tên	Vai trò	LAG	BASE	PEAK (đỉnh)
station-01	Yên Bái	thượng lưu	0.0	1.0 m	4.8 m
station-02	Sơn Tây	trung lưu	0.06	1.5 m	5.5 m
station-03	Hà Nội	hạ lưu	0.12	2.0 m	6.2 m

Bảng 3: Cấu hình ba trạm (đặt trong `docker-compose.yml`).

Các số đo phụ (`flow_rate`, `turbidity`) được suy ra tương quan với mực nước/mưa; pH thường trung tính nhưng thỉnh thoảng lệch về axit (4%) hoặc kiềm (4%) để kích hoạt luật chất lượng nước.

4.2. Chuẩn hóa tại gateway

Với mỗi bản tin, gateway kiểm tra năm trường số bắt buộc (`water_level`, `flow_rate`, `rainfall`, `turbidity`, `ph`); bản tin lỗi bị loại bỏ kèm log, không lưu. Sau đó nó tính tốc độ dâng so với số đo trước của chính trạm đó:

```

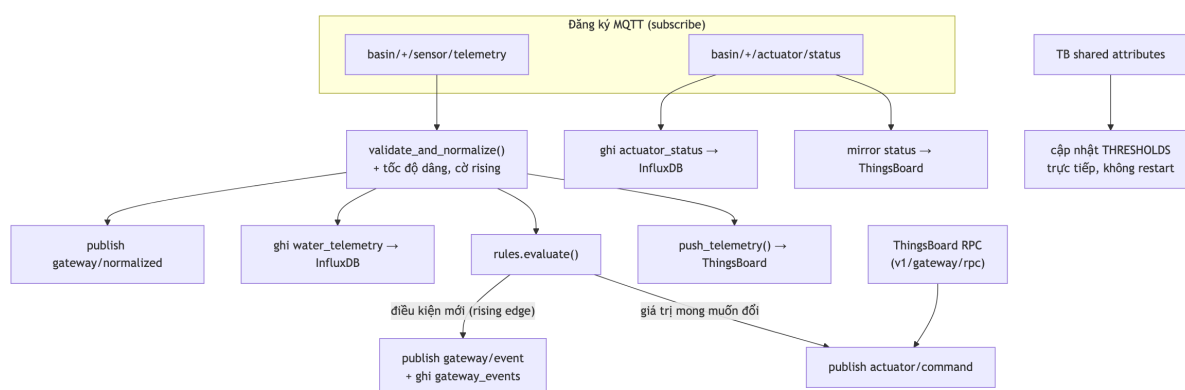
delta      = level - prev_level
rise_rate  = round(delta / dt * 60.0, 3)  # mét / phút
rising     = delta > RISE_EPSILON        # 0.005 m mỗi mẫu

```

`rising` là cờ mà luật R3 cần. Số đo đã chuẩn hóa được phát lại lên `basin/<station>/gateway/normalized` và ghi vào InfluxDB (`measurement water_telemetry`), đồng thời được đẩy lên ThingsBoard và đưa vào rule engine.

5. Rule engine

Rule engine được hiện thực trong `rules.py`: `evaluate(reading, thresholds)` trả về `(desired, active)`. Hình 5 cho thấy vị trí của rule engine trong pipeline xử lý của gateway.



Hình 5: Pipeline xử lý của gateway: chuẩn hóa, ghi InfluxDB, rule engine (event + lệnh), mirror trạng thái và cập nhật ngưỡng từ xa.

5.1. Bốn luật bắt buộc

STT	Điều kiện	Trạng thái mong muốn	Event / mức
R1	<code>water_level > 3.0</code>	pump on , gate open , board warning	<code>flood_warning</code> / warning
R2	<code>water_level > 4.0</code>	thêm siren on , board emergency	<code>flood_emergency</code> / emergency
R3	<code>rainfall > 40</code> và mực nước đang dâng	board advisory (nếu chưa cao hơn)	<code>heavy_rain</code> / advisory
R4	<code>turbidity > 100</code> hoặc <code>pH < 6</code> / <code>pH > 9</code>	chỉ cảnh báo	<code>water_quality_alert</code> / warning

Bảng 4: Bốn luật, phân mức advisory / warning / emergency.

```

def evaluate(reading, thresholds):
    level, rainfall = reading["water_level"], reading["rainfall"]
    rising = reading.get("rising", False)
    desired = {"pump": "off", "gate": "closed", "siren": "off", "board":
"normal"}
    active = {}
    # R1 / R2 - mực nước (khẩn cấp đề cảnh báo)
    if level > thresholds["level_emergency"]:
        desired.update(pump="on", gate="open", siren="on", board="emergency")
        active["flood_emergency"] = _event("flood_emergency", "emergency", ...)

```



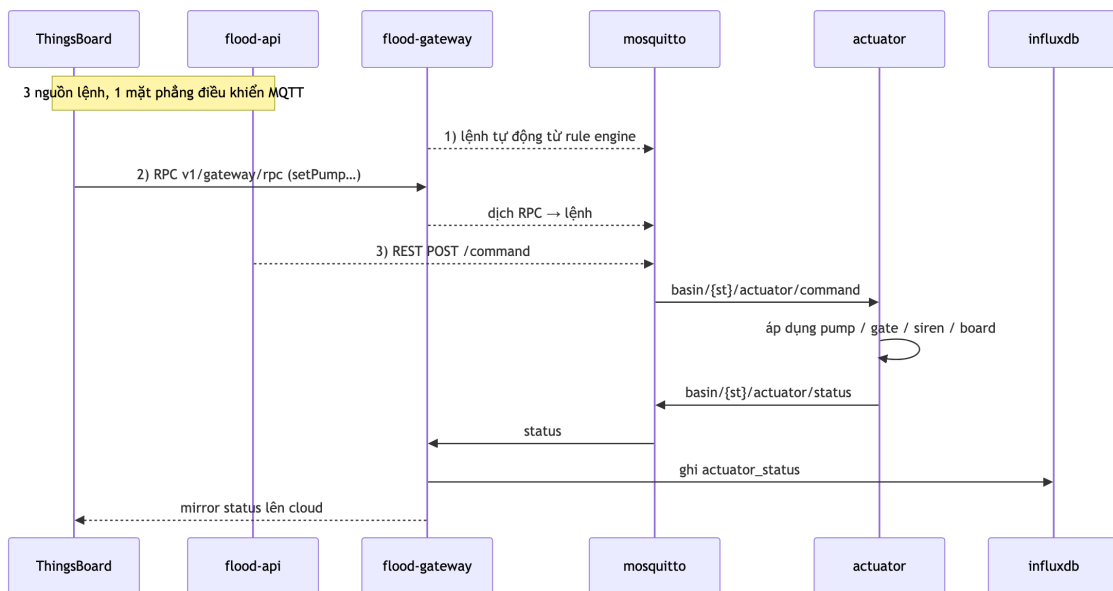
```

elif level > thresholds["level_warning"]:
    desired.update(pump="on", gate="open", board="warning")
    active["flood_warning"] = _event("flood_warning", "warning", ...)
# R3 – mưa lớn khi mực nước đang dâng
if rainfall > thresholds["rainfall_advisory"] and rising:
    if desired["board"] == "normal":
        desired["board"] = "advisory"
    active["heavy_rain"] = _event("heavy_rain", "advisory", ...)
# R4 – chất lượng nước (độ đục HOẶC pH ngoài dải)
...
return desired, active

```

6. Luồng Downlink — điều khiển thiết bị

Một lệnh có thể đến từ ba nguồn — rule engine (tự động), RPC từ ThingsBoard, hoặc gọi REST API — nhưng cả ba đều kết thúc thành cùng một bản tin MQTT trên cùng một topic `basin/<station>/actuator/command`. (Hình 6).



Hình 6: Downlink: ba nguồn lệnh hội tụ về một control plane MQTT; actuator áp dụng rồi phát lại status, gateway ghi InfluxDB và mirror lên cloud.

6.1. Thiết bị actuator

Mỗi actuator phục vụ một trạm với bốn thiết bị sau một node: **bơm**, **cửa xả/van**, **còi** và **bảng cảnh báo**. Khi nhận lệnh, nó phân giải `action` theo từng `target`, chấp nhận vài từ đồng nghĩa để một boolean của RPC, một chuỗi của REST hay một từ của gateway đều hoạt động:

target	Giá trị action chấp nhận	Trạng thái kết quả
pump	on / true / open → bật, còn lại → tắt	on off
gate	open / on / true → mở, còn lại → đóng	open closed
siren	on / true → bật, còn lại → tắt	on off
board	bất kỳ từ mức nào, lưu nguyên văn	normal .. emergency

Bảng 5: Ánh xạ lệnh → trạng thái trong `apply_command`.

Actuator phát lại toàn bộ status ngay khi có thay đổi, và heartbeat định kỳ mỗi `STATUS_INTERVAL` giây kể cả khi không đổi.

7. Tích hợp ThingsBoard (uplink + RPC)

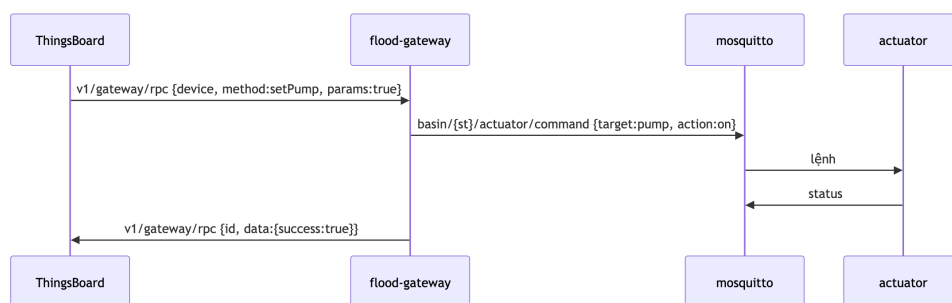
`tb_gateway.py` kết nối tới một ThingsBoard qua MQTT, dùng access token của thiết bị Gateway làm username.

Chiều	Topic	Mục đích
↑	<code>v1/gateway/connect</code>	Khai báo mỗi trạm là một sub-device.
↑	<code>v1/gateway/telemetry</code>	Đẩy telemetry từng trạm + mirror trạng thái actuator.
↑	<code>v1/gateway/attributes</code>	Gửi <code>latitude / longitude</code> tính → widget bản đồ.
↓	<code>v1/gateway/rpc</code>	Nhận lệnh server cho sub-device; trả kết quả.
↕	<code>v1/devices/me/attributes</code>	Client attrs lên; shared-attribute cập nhật xuống.

Bảng 6: Các topic Gateway API mà gateway sử dụng.

7.1. Điều khiển từ xa: RPC → lệnh cục bộ

Một nút RPC trên ThingsBoard gửi tới thiết bị trạm → TB định tuyến về gateway → `rpc_handler` dịch thành lệnh MQTT cục bộ (đúng topic mà rule engine dùng) rồi trả `{"success": true}` (Hình 7). Bốn phương thức: `setPump`, `setGate`, `setSiren` (boolean) và `setBoard` (chuỗi).

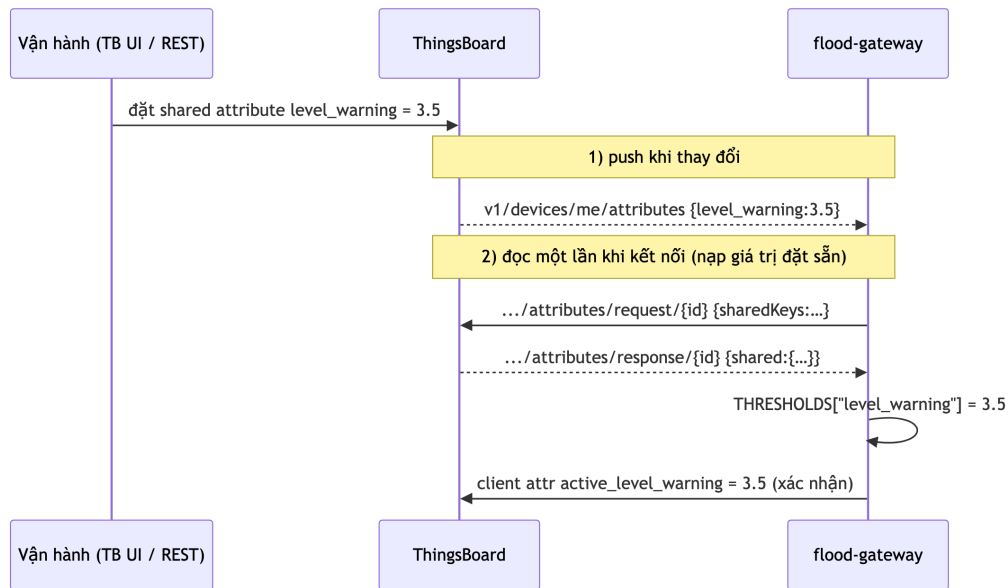


Hình 7: Trình tự RPC: nút trên ThingsBoard → gateway dịch sang lệnh MQTT → actuator → trả kết quả về cloud.

7.2. Chỉnh ngưỡng từ xa bằng shared attributes

Sáu ngưỡng cảnh báo có thể chỉnh từ thingsboard mà không cần khởi động lại, bằng cách đặt shared attribute trùng tên trên thiết bị gateway (`level_warning`, `level_emergency`, `rainfall_advisory`, `turbidity_max`, `ph_min`, `ph_max`). Một thay đổi từ xa chỉ cần tới được gateway là tick telemetry kế tiếp áp dụng cho mọi trạm.

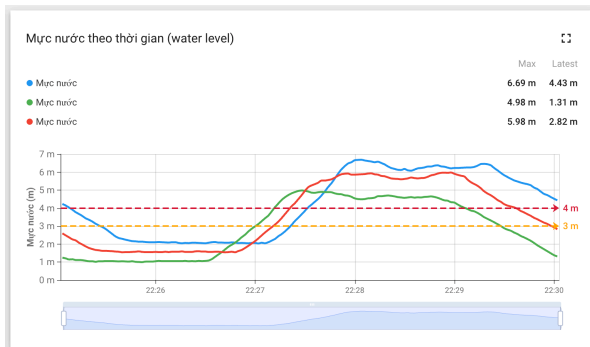
Hai cơ chế phối hợp (Hình 8): TB đẩy khi thay đổi, và gateway đọc một lần khi kết nối để nạp các giá trị người vận hành đặt trước khi gateway online. Gateway còn phản hồi giá trị đang áp dụng dưới dạng client attribute `active_<key>` để xác nhận.



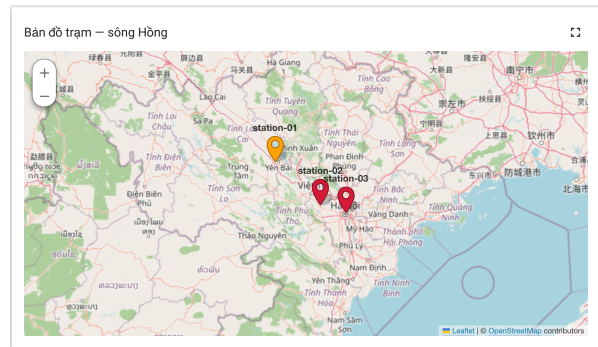
Hình 8: Chỉnh ngưỡng từ xa: push khi đổi + đọc một lần khi kết nối, và xác nhận bằng client attribute `active_*`.

7.3. Dashboard, bản đồ và alarm trên ThingsBoard

Trên đám mây, hệ thống cung cấp: biểu đồ mực nước theo thời gian cho ba trạm với đường cảnh báo 3.0/4.0 m; bản đồ OpenStreetMap với ba trạm tô màu theo mực nước (hai Hình bên dưới); các nút RPC `setPump` / `setGate` / `setSiren` cho từng trạm và một Alarm nhiều mức dựng bằng Rule Chain — `water_level > 4.0` → CRITICAL, `> 3.0` → WARNING, trở lại `≤ 3.0` → xóa alarm (Hình 11).



Hình 9: ThingsBoard — mực nước theo thời gian.



Hình 10: ThingsBoard — bản đồ ba trạm theo màu mực.

Yên Bái — Pump Pump ☐	Yên Bái — Gate Gate ☐	Yên Bái — Siren Siren ☐
Sơn Tây — Pump Pump ☐	Sơn Tây — Gate Gate ☐	Sơn Tây — Siren Siren ☐
Hà Nội — Pump Pump ☐	Hà Nội — Gate Gate ☐	Hà Nội — Siren Siren ☐

Hình 11: ThingsBoard — nút RPC điều khiển.

Alarm list						
Alarm rules						
Alarm filter For all time						
☐	Created time	Originator	Type	Severity ↓	Assignee	Status
☐	2026-06-24 09:38:56	station-02	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-24 21:17:11	station-02	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 10:38:56	station-02	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-24 10:41:59	station-01	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 02:24:50	station-01	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-24 09:08:59	station-03	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 02:36:59	station-03	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 01:39:49	station-03	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 02:34:51	station-01	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged
☐	2026-06-23 02:39:00	station-03	Water Level Flood	Warning	Unassigned	Cleared Unacknowledged

Hình 12: ThingsBoard — Bảng Alarm

8. Lưu trữ và giám sát

8.1. Lược đồ InfluxDB

Measurement	Tag	Field
water_telemetry	basin_id , station_id	water_level , flow_rate , rainfall , turbidity , ph , rise_rate
gateway_events	station_id , event_type , severity	value , threshold , action_taken
actuator_status	station_id	pump , gate , siren , board , last_command_reason

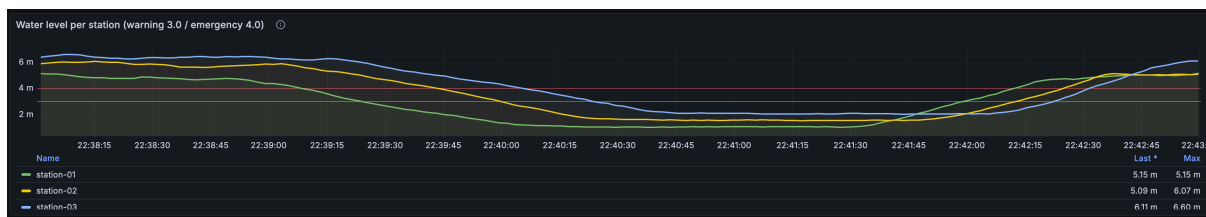
Bảng 7: Lược đồ InfluxDB — ba measurement (org navis , bucket flood).

8.2. Dashboard Grafana

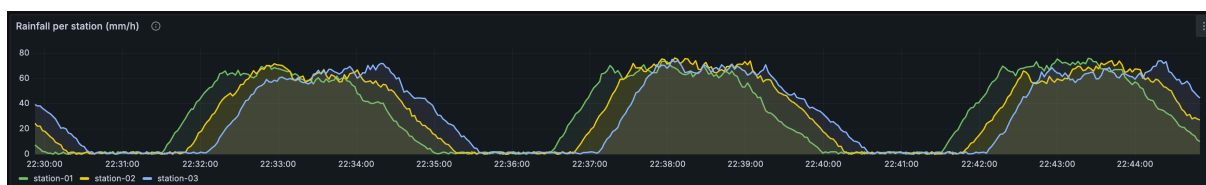
Sáu panel đáp ứng đủ yêu cầu trực quan hóa của đề bài (Bảng 8).

STT	Panel
1	Mức nước theo trạm, kèm đường warning 3.0 và emergency 4.0.
2	Lượng mưa theo trạm (mm/h), vẽ dạng đường.
3	Lưu lượng + độ đục (trục trái) và pH (trục phải), lập theo trạm.
4	Trạng thái bơm/cửa/còi/bảng dạng state timeline, lập theo trạm.
5	Số event theo mức advisory/warning/emergency theo thời gian.
6	Bảng event gần nhất: station_id , event_type , severity , action_taken .

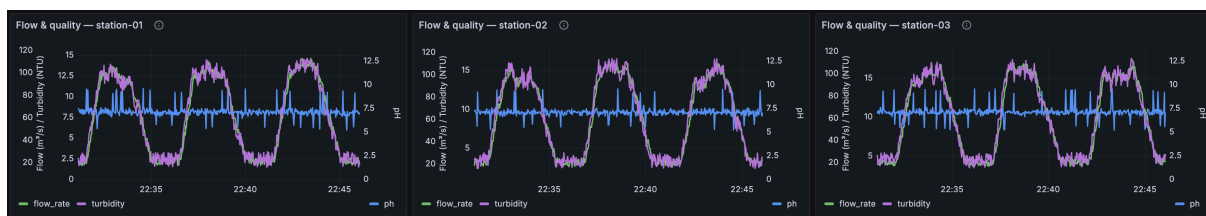
Bảng 8: Sáu panel của dashboard Grafana.



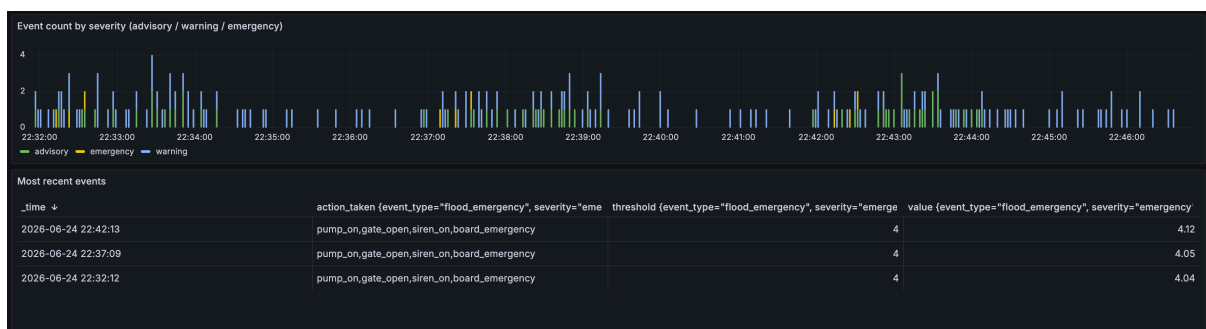
Hình 13: Mức nước + đường ngưỡng.



Hình 14: Lượng mưa



Hình 15: Chất lượng nước



Hình 16: Số event theo mức + bảng event.

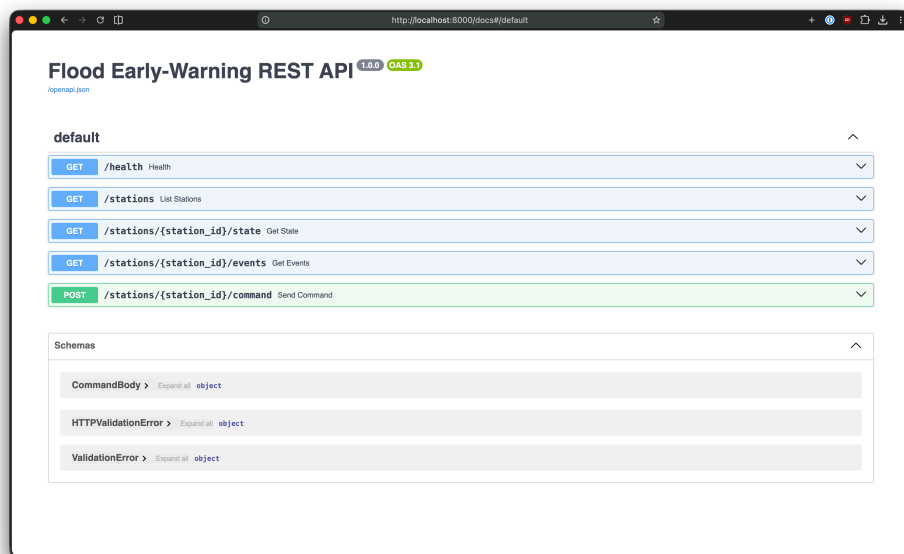
8.3. REST API (FastAPI)

Method	Endpoint	Mục đích
GET	/health	API healthcheck + InfluxDB.
GET	/stations	Mọi trạm: mức nước + bảng mới nhất.
GET	/stations/{id}/state	Telemetry + trạng thái actuator mới nhất.

Method	Endpoint	Mục đích
GET	/stations/{id}/events	Event gần đây (?limit=&hours=).
POST	/stations/{id}/command	Gửi lệnh thủ công (→ MQTT).

Bảng 9: Năm endpoint của REST API.

```
# Liệt kê trạng thái mọi trạm
curl http://localhost:8000/stations
# Gửi lệnh thủ công – cùng control plane với gateway & RPC
curl -X POST http://localhost:8000/stations/station-03/command \
-H "Content-Type: application/json" \
-d '{"target":"pump","action":"on","reason":"manual_test"}'
```



Hình 17: Tài liệu Swagger tại <http://localhost:8000/docs>.

9. Triển khai Docker Compose

Toàn bộ hệ thống khởi chạy bằng một lệnh. Mỗi service tự lập trình có Dockerfile riêng; cấu hình đọc từ biến môi trường (`.env`); các service dùng **volume** cho InfluxDB/Grafana và nói chuyện qua mạng `flood-net`.

9.1. Tham số hóa qua biến môi trường

```
sensor-station-02:
  build: ./sensor
  container_name: sensor-station-02
  env_file: .env
  environment:
    STATION_ID: station-02
    STATION_NAME: Sơn Tây
    STATION_ROLE: midstream
    STATION_LAT: "21.1480"
    STATION_LON: "105.5040"
    STATION_LAG: "0.06"      # trễ 6% chu kỳ → sóng về hạ lưu
    STATION_BASE: "1.5"     # mực nước nghỉ
    STATION_PEAK: "5.5"     # đỉnh khi mưa cực đại → mức nghiêm trọng
```

```
depends_on: [mosquitto]
networks: [flood-net]
```

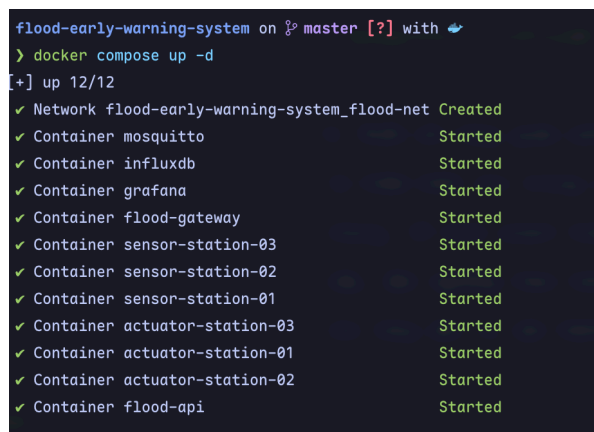
Tệp `.env` giữ cấu hình **dùng chung** (broker, InfluxDB, ngưỡng, kịch bản bão). Ví dụ một phần:

```
MQTT_BROKER=mosquitto
INFLUXDB_URL=http://influxdb:8086
INFLUXDB_ORG=navis
INFLUXDB_BUCKET=flood
LEVEL_WARNING=3.0
LEVEL_EMERGENCY=4.0
TB_HOST= # để trống = chạy edge-only, không đồng bộ cloud
```

9.2. Vận hành

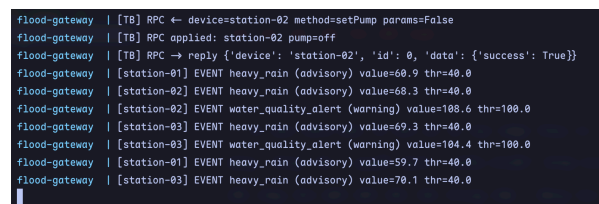
```
docker compose up -d --build # build & chạy toàn bộ stack
docker compose ps            # kiểm tra mọi service Up
docker compose logs -f flood-gateway # xem EVENT / CMD / [TB]
docker compose down          # dừng (thêm -v để xóa volume)
```

Trong một chu kỳ bão (`LOOP_PERIOD` , mặc định 300 s), cả ba trạm lần lượt vượt ngưỡng warning rồi emergency — thượng lưu `station-01` dâng trước, rồi `station-02` , cuối cùng `station-03` đỉnh cao nhất — và ta quan sát gateway phát event, điều khiển thiết bị.



```
flood-early-warning-system on % master [?] with
> docker compose up -d
[+] up 12/12
✓ Network flood-early-warning-system_flood-net Created
✓ Container mosquitto Started
✓ Container influxdb Started
✓ Container grafana Started
✓ Container flood-gateway Started
✓ Container sensor-station-03 Started
✓ Container sensor-station-02 Started
✓ Container sensor-station-01 Started
✓ Container actuator-station-03 Started
✓ Container actuator-station-01 Started
✓ Container actuator-station-02 Started
✓ Container flood-api Started
```

Hình 18: `docker compose ps` — mọi service Up.



```
flood-gateway | [TB] RPC ← device=station-02 method=setPump params=False
flood-gateway | [TB] RPC applied: station-02 pump=off
flood-gateway | [TB] RPC → reply {'device': 'station-02', 'id': 0, 'data': {'success': True}}
flood-gateway | [station-01] EVENT heavy_rain (advisory) value=60.9 thr=40.0
flood-gateway | [station-02] EVENT heavy_rain (advisory) value=68.3 thr=40.0
flood-gateway | [station-02] EVENT water_quality_alert (warning) value=108.6 thr=100.0
flood-gateway | [station-03] EVENT heavy_rain (advisory) value=69.3 thr=40.0
flood-gateway | [station-03] EVENT water_quality_alert (warning) value=104.4 thr=100.0
flood-gateway | [station-01] EVENT heavy_rain (advisory) value=59.7 thr=40.0
flood-gateway | [station-03] EVENT heavy_rain (advisory) value=70.1 thr=40.0
```

Hình 19: Log gateway — các dòng EVENT và CMD.

9.3. Khả năng phục hồi

Mọi thành phần tự kết nối lại: sensor/actuator/gateway thử lại broker mỗi 5 s; gateway chờ InfluxDB health `pass` trước khi ghi.

10. Trả lời các câu hỏi bắt buộc

1. Vì sao cảnh báo lũ cần xử lý tại edge thay vì chờ cloud? Vì độ trễ và tính sẵn sàng. Vòng “đo → quyết định → điều khiển” khép kín ngay tại trạm trong mili-giây, và vẫn chạy khi mất kết nối lên trung tâm — đúng lúc thiên tai dễ làm đứt đường truyền. Chờ cloud sẽ thêm độ trễ mạng, trong khi vài phút cảnh báo sớm vô cùng quan trọng.
2. Nhóm thiết kế topic và message format thế nào? Topic phân cấp `basin/<station>/<role>/<leaf>` (Bảng 2) cho phép tách theo trạm/vai trò và subscribe

wildcard `basin/+/sensor/telemetry`. Bản tin là JSON với bốn loại: `telemetry`, `command`, `status`, `event` — mỗi loại có tập trường cố định, kèm `station_id` và `timestamp` để truy vết.

3. Rule engine phân mức cảnh báo ra sao? Bốn luật R1–R4 (Bảng 4) ánh xạ điều kiện sang ba mức **advisory / warning / emergency**.

4. Khi mực nước vượt ngưỡng, luồng telemetry → cảnh báo → điều khiển diễn ra thế nào?

Sensor publish telemetry → gateway chuẩn hóa và tính `rise_rate` → `rules.evaluate()` sinh điều kiện active và trạng thái mong muốn → gateway phát **event** (cho điều kiện mới) và gửi **lệnh** (cho thiết bị đổi trạng thái) → actuator áp dụng và phát lại **status** → gateway ghi InfluxDB, Grafana hiển thị, đồng thời mirror lên ThingsBoard (Hình 6).

5. Gateway đóng vai trò ThingsBoard Gateway như thế nào? Nó kết nối ThingsBoard bằng MQTT với `username = access token` của thiết bị Gateway, gộp nhiều sub-device qua một kết nối: khai báo trạm (`v1/gateway/connect`), đẩy telemetry nhiều trạm cùng lúc (`v1/gateway/telemetry`), và nhận RPC (`v1/gateway/rpc`) rồi dịch thành lệnh MQTT cục bộ (Bảng 6).

6. Vì sao vẫn cần đẩy dữ liệu lên cloud dù đã xử lý ở edge? Để có tầm nhìn toàn lưu vực mà một trạm đơn lẻ không có: dashboard tập trung, bản đồ nhiều trạm, alarm và thông báo cho người vận hành, lưu trữ/đối chiếu dài hạn, điều khiển từ xa và phối hợp liên trạm.

7. Vì sao container không nên dùng localhost để gọi service khác? Vì mỗi container có **network namespace riêng** — `localhost` trở về **chính container đó**, không phải host hay container khác. Trong Docker Compose, các service phải gọi nhau **bằng tên dịch vụ** (`mosquitto`, `influxdb`) để Docker DNS phân giải đúng địa chỉ trên mạng `flood-net`.

8. Muốn mở rộng giám sát toàn thành phố trên ThingsBoard cần thay đổi gì? Thêm trạm chỉ là thêm khối env trong compose; phía cloud dùng **device profile + rule chain** dùng chung, tổ chức asset hierarchy (lưu vực → quận → trạm) để dashboard và alarm theo cấp, đặt ngưỡng theo từng trạm qua shared attributes, và phân quyền tenant/customer cho từng đơn vị vận hành.

11. Đánh giá và kết luận

11.1. Phân công công việc

Thành viên	Họ tên — MSSV	Hạng mục phụ trách
SV1	Dương Hữu Huynh	Virtual sensor, virtual actuator, thiết kế topic/bản tin.
SV2	Nông Đức Huy	Edge gateway, rule engine, ghi InfluxDB.
SV3	Bùi Phạm Sơn Hà	Tích hợp ThingsBoard + dashboard + alarm, REST API, Docker Compose, Grafana, README.

Bảng 10: Phân công cho nhóm ba thành viên.

11.2. Các yêu cầu nâng cao

Yêu cầu	Mức	Ghi chú
Bản đồ trạm + mức cảnh báo theo màu trên TB	Đã đạt	Widget map + marker tô màu theo mực nước.
Shared Attributes chỉnh ngưỡng từ xa	Đã đạt	6 ngưỡng, áp dụng nóng không restart.
Reconnect khi mất kết nối MQTT/ ThingsBoard	Đã đạt	Mọi thành phần có vòng thử lại.
Alarm nhiều mức trên ThingsBoard	Đã đạt	Rule Chain: WARNING / CRITICAL theo ngưỡng.
Dự báo xu hướng mực nước ngắn hạn	Một phần	Đã tính <code>rise_rate</code> (m/phút) — nền cho dự báo.
Unit test cho rule engine	Một phần	Engine là hàm thuần, tách I/O — sẵn sàng test; có lệnh kiểm thử nhanh.
Health check cho service trong Compose	Một phần	Gateway chờ InfluxDB health pass; chưa khai báo healthcheck trong compose.
Thông báo email/Telegram khi emergency	Hướng mở rộng	Thêm node notification vào Rule Chain TB.
Phát hiện <code>sensor_offline</code> / mất kết nối trạm	Hướng mở rộng	Theo dõi tuổi <code>prev_ts</code> trong STATE.

Bảng 11: Hiện trạng các yêu cầu nâng cao (cộng điểm).

11.3. Hạn chế và hướng phát triển

Ngưỡng hiện là toàn lưu vực (một bộ chung); có thể nâng thành theo từng trạm qua shared attributes per-device. Chưa có cảnh báo sensor offline và thông báo email/Telegram — cả hai đều khả thi bằng cách bổ sung logic ở gateway và node Rule Chain. Để giám sát quy mô thành phố, cần tổ chức asset hierarchy và phân quyền tenant.

11.4. Kết luận

Hệ thống đã hiện thực đầy đủ một hệ thống IoT hai chiều trên hạ tầng ảo hóa hoàn toàn: nhiều virtual sensor sinh telemetry tương quan theo một kịch bản bão thực tế; một edge gateway chuẩn hóa, chạy rule engine đa mức, tự động điều khiển thiết bị, ghi InfluxDB và hiển thị Grafana tại biên; đồng thời đóng vai trò ThingsBoard Gateway để đẩy telemetry và nhận RPC từ đám mây. Toàn bộ khởi chạy bằng một lệnh Docker Compose. Qua đó, đề tài làm rõ giá trị của xử lý thời gian thực tại biên cho các tình huống khẩn cấp, kết hợp với giám sát toàn lưu vực trên đám mây.

Toàn bộ source code, tài liệu được lưu trữ tại: <https://github.com/betty2310/flood-warning-system>^o

Danh mục hình

Hình 1	Kiến trúc tổng thể: thiết bị ảo mỗi trạm, broker MQTT, edge gateway, InfluxDB, Grafana, REST API và ThingsBoard.	4
Hình 2	Cây topic dưới mỗi trạm và chiều đi của bản tin.	5
Hình 3	Trình tự uplink: telemetry → chuẩn hóa → lưu trữ + đẩy cloud + rule engine.	6
Hình 4	Lượng mưa theo hình thang trong một chu kỳ bão (lặng 30% → dâng 15% → cao nguyên 30% → rút 25%).	7
Hình 5	Pipeline xử lý của gateway: chuẩn hóa, ghi InfluxDB, rule engine (event + lệnh), mirror trạng thái và cập nhật ngưỡng từ xa.	8
Hình 6	Downlink: ba nguồn lệnh hội tụ về một control plane MQTT; actuator áp dụng rồi phát lại status, gateway ghi InfluxDB và mirror lên cloud.	9
Hình 7	Trình tự RPC: nút trên ThingsBoard → gateway dịch sang lệnh MQTT → actuator → trả kết quả về cloud.	10
Hình 8	Chỉnh ngưỡng từ xa: push khi đổi + đọc một lần khi kết nối, và xác nhận bằng client attribute <code>active_*</code>	11
Hình 9	ThingsBoard — mực nước theo thời gian.	11
Hình 10	ThingsBoard — bản đồ ba trạm theo màu mức.	11
Hình 11	ThingsBoard — nút RPC điều khiển.	11
Hình 12	ThingsBoard — Bảng Alarm	12
Hình 13	Mực nước + đường ngưỡng.	13
Hình 14	Lượng mưa	13
Hình 15	Chất lượng nước	13
Hình 16	Số event theo mức + bảng event.	13
Hình 17	Tài liệu Swagger tại <code>http://localhost:8000/docs</code>	14
Hình 18	<code>docker compose ps</code> — mọi service Up.	15
Hình 19	Log gateway — các dòng EVENT và CMD.	15

Danh mục bảng

Bảng 1	Bản đồ dịch vụ — 11 container trong một dự án Docker Compose.	4
Bảng 2	Năm topic tại biên cùng cặp producer → consumer.	5
Bảng 3	Cấu hình ba trạm (đặt trong <code>docker-compose.yml</code>).	7
Bảng 4	Bốn luật, phân mức advisory / warning / emergency.	8
Bảng 5	Ánh xạ lệnh → trạng thái trong <code>apply_command</code>	9
Bảng 6	Các topic Gateway API mà gateway sử dụng.	10
Bảng 7	Lược đồ InfluxDB — ba measurement (org <code>navis</code> , bucket <code>flood</code>).	12
Bảng 8	Sáu panel của dashboard Grafana.	12
Bảng 9	Năm endpoint của REST API.	13
Bảng 10	Phân công cho nhóm ba thành viên.	16
Bảng 11	Hiện trạng các yêu cầu nâng cao (cộng điểm).	17